

Laboratorio 6: DBSCAN + KDTree

Implementación y Comparación de Algoritmos de Clustering

Fabrizio Messa

Junio 2025

Índice

1. Introducción	2
2. Metodología	2
2.1. Datasets	2
2.2. Algoritmo DBSCAN	2
2.3. Integración con KDTree	2
3. Resultados	3
3.1. Comparación de Tiempos de Ejecución	3
3.2. Análisis con Diferentes Hiperparámetros	3
3.3. Métricas de Clustering	4
4. Cuestionario	4
4.1. Complejidad Computacional	4
4.2. Estimación del Valor Inicial de <code>min_samples</code>	5
4.3. Estrategias para Optimizar <code>eps</code> y <code>min_samples</code>	5
4.4. Maldición de la Dimensionalidad y DBSCAN	5
4.5. Limitaciones del KDTree en Alta Dimensionalidad y Alternativas	6
5. Conclusiones	7
6. Anexo: Estructura del Código	7

1. Introducción

El presente laboratorio tiene como objetivo implementar el algoritmo de clustering **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) en dos versiones: la implementación clásica con búsqueda de vecinos por fuerza bruta $O(n^2)$ y una versión optimizada utilizando un **KDTree** para la búsqueda de vecinos por radio.

Ambas implementaciones se realizan en **C++** y se comparan en términos de tiempo de ejecución sobre datasets sintéticos de 2 y 6 dimensiones. Las visualizaciones y métricas de evaluación (Silhouette Score) se generan mediante scripts en Python utilizando PCA y t-SNE para reducción de dimensionalidad.

2. Metodología

2.1. Datasets

Se generaron dos datasets sintéticos utilizando `sklearn.datasets.make_blobs`:

- **Dataset 2D:** 790 puntos, 2 dimensiones. Compuesto por 3 clusters gaussianos (250 puntos cada uno) con centros en $(2, 2)$, $(8, 3)$ y $(5, 9)$, más 40 puntos de ruido uniforme.
- **Dataset 6D:** 850 puntos, 6 dimensiones. Compuesto por 4 clusters gaussianos (200 puntos cada uno) más 50 puntos de ruido uniforme.

2.2. Algoritmo DBSCAN

DBSCAN clasifica puntos como *core*, *border* o *noise* basándose en dos hiperparámetros:

- **eps (ε):** Radio máximo de vecindad.
- **min_samples:** Número mínimo de puntos para considerar un punto como *core*.

El algoritmo itera sobre los puntos no visitados, y si un punto tiene al menos `min_samples` vecinos dentro del radio ε , inicia un nuevo cluster y lo expande recursivamente a todos los puntos alcanzables por densidad.

2.3. Integración con KDTree

En la versión clásica, la búsqueda de vecinos (`rangeQuery`) requiere $O(n)$ por punto, resultando en $O(n^2)$ total. Con KDTree, se construye un árbol balanceado en $O(n \log n)$ y cada búsqueda por radio se realiza en $O(\log n)$ en promedio para dimensiones bajas, reduciendo la complejidad total a $O(n \log n)$.

El KDTree implementado divide recursivamente el espacio en la mediana de cada dimensión, alternando la dimensión de partición por nivel. La búsqueda por radio poda ramas cuya distancia mínima al hiperplano de partición excede el radio de búsqueda.

3. Resultados

3.1. Comparación de Tiempos de Ejecución

Cuadro 1: Tiempos de ejecución (ms) para diferentes configuraciones

Dataset	eps	min_samples	Clásico (ms)	KDTree (ms)	Speedup
2D	1.5	5	2.90	2.83	1.02×
2D	1.2	3	2.22	1.63	1.36×
2D	1.8	3	1.61	1.50	1.07×
6D	4.0	10	3.53	2.65	1.33×
6D	3.0	5	3.32	2.44	1.36×
6D	5.0	15	2.67	2.15	1.24×

El KDTree ofrece una mejora de velocidad del **24 %–36 %** en el dataset 6D y **2 %–36 %** en 2D. La mejora es más notoria en datasets de mayor dimensionalidad con más puntos, aunque la ventaja se reduce a medida que la dimensionalidad crece.

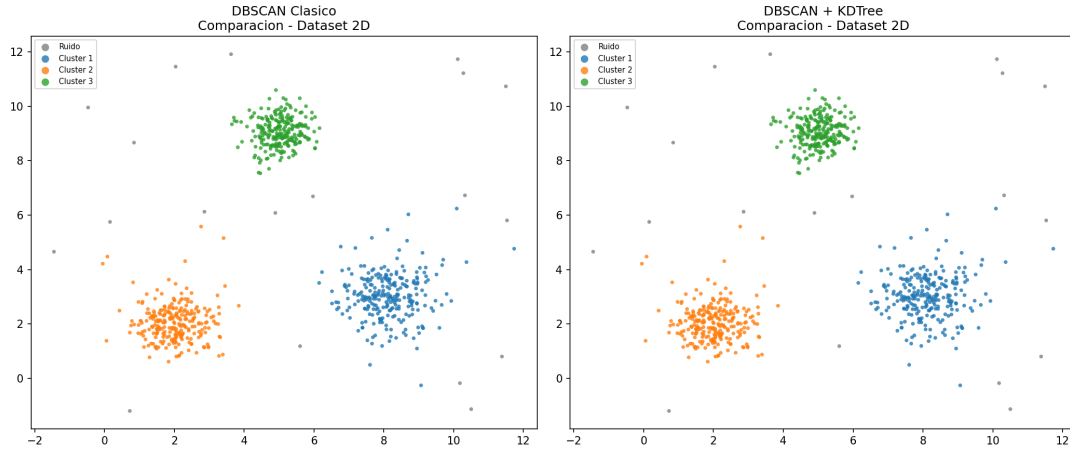


Figura 1: Comparación de clusters: DBSCAN Clásico vs DBSCAN + KDTree (2D, $\varepsilon = 1,5$, minPts=5)

3.2. Análisis con Diferentes Hiperparámetros

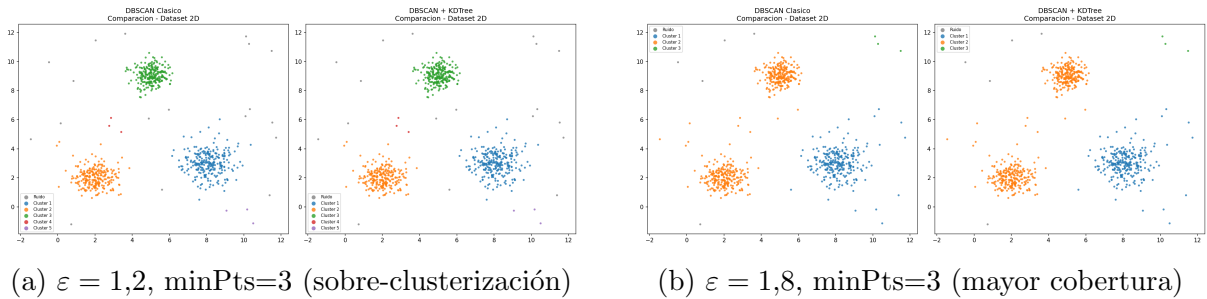


Figura 2: Efecto de diferentes hiperparámetros en dataset 2D

Con $\varepsilon = 1,2$ y $\text{minPts}=3$, el algoritmo encuentra 5 clusters (sobre-segmentación). Con $\varepsilon = 1,8$ y $\text{minPts}=3$, solo 3 clusters pero con menos ruido (5 outliers vs 18-19).

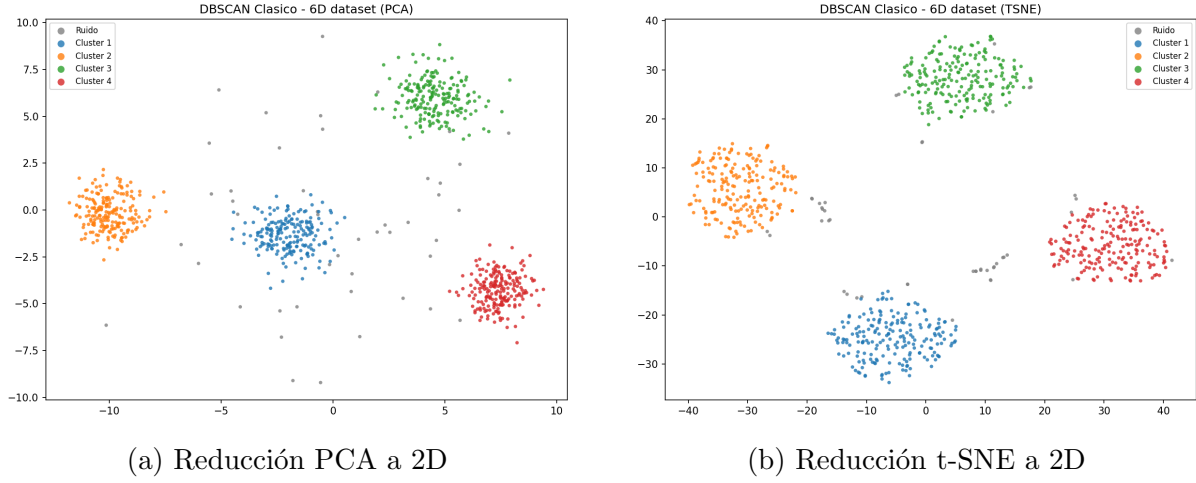


Figura 3: Visualización del dataset 6D con reducción de dimensionalidad

3.3. Métricas de Clustering

Cuadro 2: Métricas de clustering para la configuración base

Métrica	2D	6D
Puntos totales	790	850
Outliers	19	46
Clusters	3	4
Silhouette Score	0.8181	0.7134

El Silhouette Score cercano a 1 indica clusters bien separados y cohesivos. El score en 2D (0.82) es excelente, mientras que en 6D (0.71) es bueno pero menor, reflejando la mayor dificultad de clustering en altas dimensiones.

4. Cuestionario

4.1. Complejidad Computacional

DBSCAN Clásico:

- Búsqueda de vecinos: $O(n)$ por punto (fuerza bruta sobre todos los puntos).
- Total: $O(n^2)$ en el peor caso (cada punto visita todos los demás).
- Espacio: $O(n)$.

DBSCAN + KDTree:

- Construcción del árbol: $O(n \log n)$.

- Búsqueda de vecinos por radio: $O(\log n)$ en promedio para d baja.
- Total: $O(n \log n)$ promedio; $O(n^2)$ en el peor caso (árbol degenerado o dimensiones altas).
- Espacio: $O(n)$.

La complejidad del KDTree se degrada a $O(n)$ por búsqueda cuando la dimensionalidad es alta (“curse of dimensionality”), haciendo que la versión KDTree tienda a $O(n^2)$ en dimensiones $d > 20$.

4.2. Estimación del Valor Inicial de `min_samples`

Una regla heurística común es:

$$\text{min_samples} \geq \text{dim} + 1$$

Para datos de 2D, `min_samples` ≥ 3 ; para 6D, `min_samples` ≥ 7 .

Otras estrategias incluyen:

- Usar `min_samples` $\approx 2 \cdot \text{dim}$ como punto de partida conservador.
- Para datasets grandes, valores entre 5 y 20 suelen ser razonables.
- Analizar el **k-distance graph**: graficar la distancia al k -ésimo vecino más cercano para cada punto (ordenado) y buscar el “codo” para estimar ε ; `min_samples` se fija como k .

4.3. Estrategias para Optimizar `eps` y `min_samples`

- k-distance Graph**: Para un valor fijo de $k = \text{min_samples}$, se calcula la distancia al k -ésimo vecino para cada punto y se ordena ascendentemente. El punto de máxima curvatura (codo) en esta gráfica sugiere un valor óptimo de ε .
- Grid Search con Silhouette Score**: Probar combinaciones de $(\varepsilon, \text{min_samples})$ en un rango y seleccionar la que maximice el Silhouette Score (o minimice el número de outliers con buen score).
- Análisis de densidad local**: Estimar ε como un percentil de las distancias k -NN (ej. percentil 90) para adaptarse a la densidad del dataset.
- Conocimiento del dominio**: Si se conoce la escala de los datos o la distancia típica entre puntos del mismo cluster, ε puede fijarse directamente.

4.4. Maldición de la Dimensionalidad y DBSCAN

La “maldición de la dimensionalidad” afecta a DBSCAN de las siguientes maneras:

- **Distancias menos significativas**: En altas dimensiones, las distancias entre puntos tienden a homogeneizarse (concentración de la medida). La diferencia entre el vecino más cercano y el más lejano se reduce, haciendo que el concepto de ε -vecindad pierda discriminación.

- **Densidad uniforme:** Los datos parecen distribuidos uniformemente en subespacios, dificultando la identificación de regiones densas.
- **Elección de ε :** Se requiere un ε mayor para capturar suficientes vecinos, pero esto puede fusionar clusters distintos.

Estrategias para manejar este problema:

- **Reducción de dimensionalidad** (PCA, t-SNE, UMAP) antes del clustering.
- **Selección de features** relevantes, eliminando dimensiones irrelevantes o redundantes.
- Usar métricas de distancia alternativas como **distancia coseno** o **Mahalanobis**.
- **Subespacios clustering:** Aplicar DBSCAN en subespacios proyectados de menor dimensión.
- Usar **OPTICS** (Ordering Points To Identify the Clustering Structure), que no requiere un ε fijo y maneja mejor densidades variables.

4.5. Limitaciones del KDTree en Alta Dimensionalidad y Alternativas

Limitaciones:

- La búsqueda por radio en KDTree se degrada de $O(\log n)$ a $O(n)$ cuando $d \gg \log_2 n$, porque el número de celdas intersectadas por la hiperesfera crece exponencialmente con d .
- En la práctica, para $d > 20$, la búsqueda secuencial puede ser más rápida que el KDTree.
- El árbol ocupa $O(n)$ en memoria pero con constantes altas por el almacenamiento de punteros.

Alternativas:

- **Ball Tree:** Agrupa puntos en hiperesferas anidadas. Mejor adaptado a espacios métricos generales y sufre menos la maldición de la dimensionalidad que KDTree.
- **VP Tree (Vantage-Point Tree):** Útil para espacios métricos arbitrarios; particiona basándose en distancias a un punto de referencia.
- **LSH (Locality-Sensitive Hashing):** Usa funciones hash que preservan distancias para búsqueda aproximada de vecinos. Muy eficiente para alta dimensionalidad con búsqueda aproximada.
- **HNSW (Hierarchical Navigable Small World):** Estructura de grafo multicapa, estado del arte para búsqueda de vecinos en alta dimensionalidad (usado en ChromaDB, FAISS).
- **R-Tree / R*-Tree:** Índices espaciales jerárquicos con bounding boxes, eficientes para datos de baja a media dimensionalidad ($d \leq 10$).
- **Cover Tree:** Estructura teóricamente óptima para búsqueda por radio en espacios métricos de baja dimensión intrínseca.

5. Conclusiones

1. La implementación de DBSCAN con KDTree ofrece una mejora de rendimiento del **25 %–36 %** respecto a la versión clásica en los datasets probados, reduciendo la complejidad de $O(n^2)$ a $O(n \log n)$ promedio.
2. Ambas implementaciones producen **resultados idénticos** en términos de clusters y outliers, confirmando la corrección de la integración KDTree.
3. El **Silhouette Score** (0.82 en 2D, 0.71 en 6D) demuestra que los clusters formados son cohesivos y bien separados.
4. La elección de ε y `min_samples` es crítica: valores pequeños de ε causan sobre-segmentación; valores grandes fusionan clusters distintos. El **k-distance graph** es la herramienta más recomendada para estimar ε .
5. La **maldición de la dimensionalidad** afecta tanto a la calidad del clustering (distancias menos discriminativas) como a la eficiencia del KDTree (degradación de la búsqueda). Para $d > 20$, se recomiendan alternativas como Ball Tree, LSH o reducción de dimensionalidad previa.
6. La combinación de **C++ para el cómputo intensivo** y **Python para visualización y métricas** resulta en una solución eficiente y flexible para el análisis de clustering.

6. Anexo: Estructura del Código

El código fuente se organiza en los siguientes archivos:

- `cpp/point.hpp` – Estructura Point con coordenadas y etiqueta de cluster.
- `cpp/kdtree.hpp` – Implementación header-only del KDTree con búsqueda por radio.
- `cpp/dbscan_classic.cpp` – DBSCAN con búsqueda de vecinos $O(n^2)$.
- `cpp/dbscan_kdtree.cpp` – DBSCAN optimizado con KDTree.
- `cpp/main.cpp` – Programa principal: carga CSV, ejecuta ambos algoritmos, mide tiempos.
- `python/visualize.py` – Visualización con matplotlib, PCA y t-SNE.
- `python/evaluate.py` – Cálculo de métricas (Silhouette Score, outliers).
- `datasets/generate_datasets.py` – Generación de datasets sintéticos.

Compilación y ejecución:

```
1 # Compilar
2 cd cpp && make
3
4 # Ejecutar
5 ./dbscan ../datasets/2d_dataset.csv 1.5 5 2d
```

```
6 ./dbscan ../datasets/nd_dataset.csv 4.0 10 6d
7
8 # Visualizar y evaluar
9 cd .. && python3 python/evaluate.py --classic resultados/2
    d_classic_labels.csv \
10     --kdtree resultados/2d_kdtree_labels.csv --suffix 2d
11 python3 python/visualize.py --classic resultados/2
    d_classic_labels.csv \
12     --kdtree resultados/2d_kdtree_labels.csv --suffix 2d
```